

MAP Nexus — Admin UI & Access-Control Specification

Design C Mission Control Dashboard + Design B Administration Console + Design A Capability Cards

Version 1.0 | MAP_V3 | Design specification for OpenCode implementation review

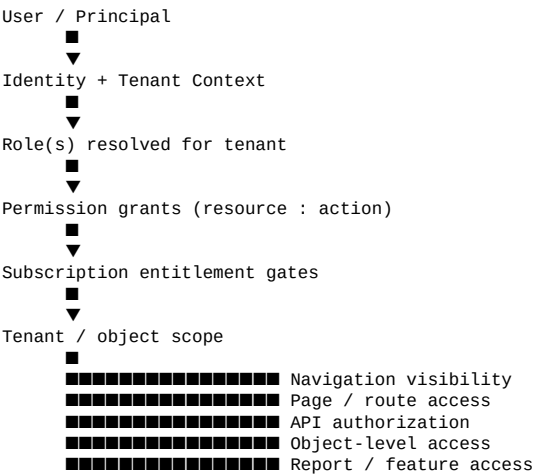
1. Executive decision

- Design B — Administration Console: persistent left rail + content/detail pane.
- Design C — Mission Control: /administration landing page is a dashboard.
- Design A — Capability Cards: dashboard cards provide status and shortcuts.
- Single access model: Identity → Role → Permission → Subscription Entitlement → Tenant/Object Scope.
- Tenant dropdown/switcher is now specified here; it was deferred during the earlier concept exercise.

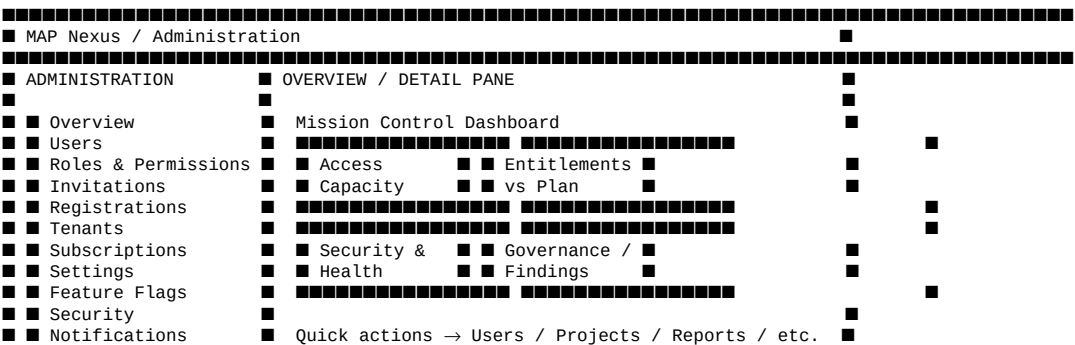
2. Non-negotiable principles

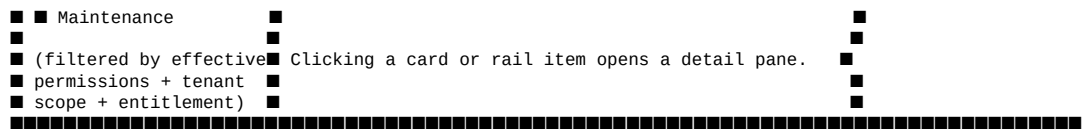
- One authoritative permission model.
- UI visibility is not security; backend/API authorization is authoritative.
- Tenant isolation is enforced server-side and at query/object scope.
- Super Admin is platform/global; Tenant Admin is tenant-scoped.
- Subscription entitlements are independent of RBAC permissions.
- Do not create separate permission systems for reports, menus, pages and objects.
- Preserve working functionality unless explicitly changed.
- Do not add duplicate routes, services, tables or permission stores.
- System roles are protected; custom roles are tenant-scoped.
- Usage limits are plan-driven and enforced server-side.
- Security-sensitive administrative actions are auditable.

3. High-level architecture



4. Administration shell





Persistent rail

- Overview
- Users
- Roles & Permissions
- Invitations
- Registrations
- Tenants
- Subscriptions
- Settings
- Feature Flags
- Security
- Notifications
- Maintenance

Visibility is calculated from effective permission + entitlement + tenant scope.

Detail pane

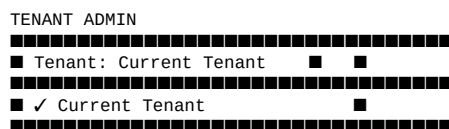
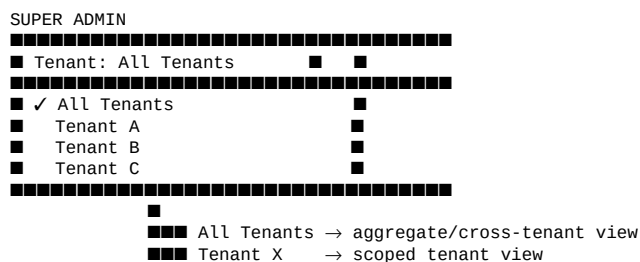
One administration section is active at a time. Breadcrumb: Administration → Section → Detail.

5. Mission Control dashboard

- Governance posture: status, findings/alerts and Audit/Governance shortcut.
- Access capacity: users, projects and connections used/allowed, with relevant actions.
- Entitlements vs plan: enabled, limited and unavailable capabilities.
- Security & health: API/database/notification health, sessions, failed logins and security shortcut.
- Role & permission matrix.
- Pending registrations / operational alerts where permitted.
- Quick actions limited to permitted actions.

Cards are shortcuts, not a second navigation hierarchy.

6. Tenant dropdown / tenant switcher



Tenant Admin must not select or inspect another tenant.

- Super Admin: All Tenants plus individual tenants; switching changes working/view context, not the user's role.
- All-Tenants mode clearly identifies aggregate/cross-tenant data.
- Tenant Admin: current tenant only; no cross-tenant selector and no API/route escape.
- Changing tenant refreshes tenant-specific dashboard and administration data.

- Global platform data remains visible only where permission allows it.

7. Role model

Role	Scope	Administration	Tenant access
Super Admin	Platform/global	Full administration; tenant lifecycle; plans; global security; cross-tenant views	All tenants
Tenant Admin	Tenant	Tenant users, roles, permissions within allowed boundary, invitations, settings, operations	Own tenant only
Migration Lead	Tenant	No general administration	Own tenant; migration/validation operations
Data Analyst	Tenant	No general administration	Own tenant; reporting/validation
Team Member	Tenant	No general administration	Own tenant; assigned/allowed objects
Viewer	Tenant	Read-only	Own tenant; read-only objects/reports

8. Permission model

Use one resource/action capability model across navigation, routes, APIs, objects and reports.

users:view
users:create
users:edit
users:disable

projects:view
projects:create
projects:edit
projects:delete

reports.validation:view
reports.validation:export

Resource	Example actions	Typical UI surfaces
users	view, create, edit, disable	Users card, Users page, API
roles	view, create, edit, assign	Roles & Permissions
invitations	view, create, revoke	Invitations
registrations	view, convert	Registrations (Super Admin boundary)
tenants	view, create, suspend, edit	Tenants (Super Admin boundary)
projects	view, create, edit, delete	Projects
systems/connections	view, create, test, edit	Systems / Connections
reports	view, export	Reports
security	view, manage permitted policy	Security
maintenance	view, execute permitted operation	Maintenance
subscriptions	view, manage	Subscription / Plans

9. Subscription entitlements

RBAC answers who may perform an action. Subscription entitlement answers whether the tenant's plan includes the capability.

Effective access = Identity AND Role permission AND Subscription entitlement AND Tenant/object scope

Layer	Purpose
RBAC permission	Answers: is this principal allowed to perform this action?
Subscription entitlement	Answers: is this feature included in the tenant's plan?
Tenant/object scope	Answers: which tenant/object can the principal access?
UI visibility	UX projection only; never the security boundary.
API enforcement	Authoritative security boundary; must enforce the same model.

10. Super Admin vs Tenant Admin

Super Admin

- All-tenant visibility and tenant lifecycle.
- Plans/subscriptions and global security/platform settings.
- System roles and permission templates.
- User lock/unlock and cross-tenant operational/governance information.
- Tenant switching.

Tenant Admin

- Own-tenant users, invitations, roles within boundary, projects, systems/connections, settings and reports.
- No other-tenant access, global permission changes, Super Admin changes or cross-tenant data.

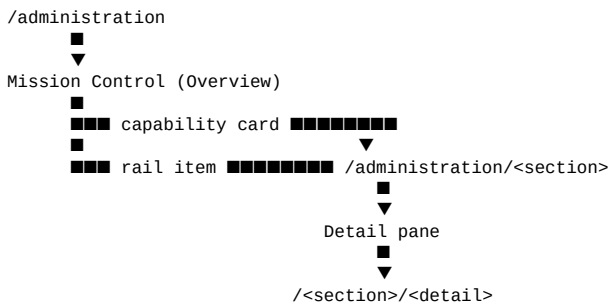
11. Role & permission management

- System roles: Super Admin, Tenant Admin, Migration Lead, Data Analyst, Team Member, Viewer.
- System roles are protected.
- Custom roles are tenant-scoped and cannot grant outside plan or protected global capabilities.
- Roles UI shows name, scope, users, capabilities, protected/system status and restrictions.

12. Users and subscription capacity

- Show usage against plan limits.
- Enforce limits server-side.
- Explain unavailable actions.
- Tenant Admin sees only own tenant; Super Admin can inspect tenant capacity.

13. Navigation and route rules



Navigation, routes, APIs and database/query layers must enforce the same effective access model. Hidden menus are not security.

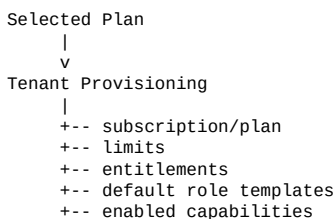
14. Dashboard data and charts

- KPI/usage cards.
- Compact progress bars.
- Small meaningful trend charts.
- Status indicators and findings/alerts.
- Consistent refresh/currentness indicator.
- Avoid excessive API calls.

15. Security and audit requirements

- Audit user lifecycle, role/permission changes, invitations, tenant lifecycle, subscriptions, security policy and maintenance actions.
- Where supported, record actor, tenant/context, action, target, timestamp and outcome.

16. Onboarding and subscription provisioning



Use the canonical subscription/entitlement source; do not duplicate plan logic.

17. Implementation constraints for OpenCode

- Inspect current navigation, permissions, roles, subscription/plan data, tenant context and route guards before changing code.
- Reuse existing services, tables and components.

- Identify conflicting/duplicate permission or role stores.
- If backend authorization or schema is affected, produce a file-level implementation plan for approval.
- Preserve working APIs; do not add migrations merely to make the UI appear to work.
- Do not weaken TypeScript, authentication or authorization.
- Do not commit unrelated cleanup.
- Keep changes within the agreed work package.
- Verify build, targeted tests, backend tests and representative access cases.

18. Acceptance criteria

- Administration opens on a Mission Control dashboard.
- Persistent Administration rail is available on every administration screen.
- Rail, cards, routes and API authorization use one authoritative capability/permission model.
- Super Admin can use a tenant filter/switcher with an All Tenants option.
- Tenant Admin is restricted to the current tenant and cannot switch to another tenant.
- Dashboard cards expose useful status and shortcuts without becoming a second navigation system.
- A card or rail item opens the corresponding console detail pane.
- Subscription entitlements can hide, disable or mark unavailable capabilities without bypassing server-side authorization.
- No frontend-only role check is treated as sufficient security.
- Global/platform controls are unavailable to Tenant Admin even if a frontend route is manually opened.
- System roles are protected; custom roles must not grant capabilities outside the tenant/plan boundary.
- Usage limits (users/projects/connections/etc.) are visible where relevant and enforced server-side.
- Audit events exist for security-sensitive administration actions.
- The implementation preserves existing working routes and services unless explicitly changed by this specification.
- No new duplicate permission store is introduced.

19. Implementation sequence

Stage A — Access-control foundation

- Normalize roles.
- Confirm permission source.
- Confirm tenant/object scope.
- Confirm entitlement source.

Stage B — Administration shell

- Persistent rail.
- Mission Control landing.
- Capability cards.
- Detail-pane navigation.

Stage C — Tenant context

- Super Admin tenant switcher.
- All Tenants mode.
- Tenant Admin locked scope.
- Context propagation and refresh.

Stage D — Dashboard data

- Capacity.
- Entitlements.
- Governance.

- Security/health.
- Operational alerts.
- Quick actions.

Stage E — Admin sections

- Users.
- Roles & Permissions.
- Invitations.
- Registrations.
- Tenants.
- Subscriptions.
- Settings.
- Feature Flags.
- Security.
- Notifications.
- Maintenance.

Stage F — Verification

- Build.
- Tests.
- RBAC matrix.
- Tenant isolation.
- Entitlement checks.
- Audit evidence.

Do not implement all stages in one uncontrolled change. Review and verify each stage before the next.

20. Explicit non-goals

- Billing-provider implementation.
- Payment checkout flow.
- Entire application-shell redesign.
- Migration engine/rule logic.
- Unrelated frontend cleanup.
- New permission stores.
- Speculative AI administration features.

21. OpenCode handover instruction

Treat this document as the authoritative Admin UI and Access-Control design baseline for the next implementation work package.

- Reconcile against live MAP_V3 codebase.
- Report reusable components/services/tables.
- Report role/permission conflicts.
- Report genuinely required schema changes.
- Report affected routes.
- Produce a staged implementation plan.
- Do not begin broad implementation until reconciliation and staged plan are reviewed.